

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3.6.6

UPGRADE TO 3.0

 Copy as Markdown

Upgrade to 3.0

NOTE

The new authentication module has separate migration documentation. This guide covers only core framework changes.

Requirements

- Dart SDK: 3.8.0+
- Flutter SDK: 3.32.0+

Breaking changes summary

- **Web server:** Relic framework integration (custom routes need updates)
- **Session.request:** New property to access HTTP request from endpoints
- **Authentication:** `AuthenticationInfo` changes, `session.authenticated` now synchronous
- **Enum serialization:** Default changed from `byIndex` to `byName`
- **Model changes:** `SerializableEntity` removed, YAML keywords updated

Updating your project

Update the CLI

Update the Serverpod command line interface to the latest version:

```
dart pub global activate serverpod_cli
```



Ask AI

Verify the installed version:

```
serverpod version
```

Update pubspec.yaml files

Update the `pubspec.yaml` files in your `server`, `client`, and `flutter` directories. Change all Serverpod package versions to `3.0.0` (or later 3.x version).

Update the Dart SDK constraint in all `pubspec.yaml` files:

```
environment:  
  sdk: '≥3.8.0 <4.0.0'
```

Update the Dockerfile

Update the Dart version in your project's `Dockerfile`:

```
FROM dart:3.8 AS build  
  
...
```

Run updates

After updating your `pubspec.yaml` files, run these commands in each package directory:

```
dart pub upgrade
```

Then regenerate your project in your `server` directory:

```
serverpod generate
```

Create and apply migrations

A database migration is required for Serverpod 3.0. The session log table now stores authenticated user IDs as `String` instead of `int` to support non-numeric identifiers (e.g., `UUID`s).

Create and apply the migration:

```
serverpod create-migration  
dart run bin/main.dart --apply-migrations
```

Migration checklist

1. Update CLI, pubspec files, and Dockerfile (see above)
2. Update custom Route classes (see Web Server section)
3. Update enum serialization strategy (see Enum section)
4. Replace `SerializableEntity` with `SerializableModel` in custom models
5. Update custom `AuthenticationInfo` usage (see Authentication section)
6. Test and deploy

Web server (Relic framework)

Serverpod 3.0 integrates the [Relic framework](#) for web server functionality, bringing it out of experimental status. This provides improved performance, better request/response handling, and built-in WebSocket support.

Route.handleCall signature

Aspect	Serverpod 2.x	Serverpod 3.0
Request	<code>HttpRequest request</code>	<code>Request request</code>
Return	<code>Future<bool></code>	<code>FutureOr<Result></code>
Response	<code>return true</code>	<code>return Response.ok()</code>

Before:

```
import 'dart:io';

class MyRoute extends Route {
  @override
  Future<bool> handleCall(Session session, HttpRequest request) async {
    request.response.write('<html><body>Hello</body></html>');
    return true;
  }
}
```

After:

```
class MyRoute extends Route {
  @override
  FutureOr<Result> handleCall(Session session, Request request) async {
    return Response.ok(
      body: Body.fromString('<html><body>Hello</body></html>', mimeType: MimeType.html),
    );
  }
}
```

```

    };
  }
}

```

Changes:

- `HttpRequest` → `Request` (from Relic)
- `Future<bool>` → `FutureOr<Result>`
- `request.remoteIpAddress` → `request.remoteInfo` (both return `String`)
- `request.headers.value('name')` → `request.headers['name']`
- Return `Response` instead of `bool`

Widget class names

The web server widget classes have been reorganized for better clarity. Legacy class names are deprecated but still available for backward compatibility.

Old (Deprecated)	New
<code>AbstractWidget</code>	<code>WebWidget</code>
<code>Widget</code>	<code>TemplateWidget</code>
<code>WidgetList</code>	<code>ListWidget</code>
<code>WidgetJson</code>	<code>JsonWidget</code>
<code>WidgetRedirect</code>	<code>RedirectWidget</code>

The `WidgetRoute` class remains unchanged and continues to be the base class for web routes.

Before:

```
Future<Widget> build(...) ⇒ Widget(name: 'page')..values = {...};
```

After:

```
Future<WebWidget> build(...) ⇒ TemplateWidget(name: 'page', values: {...});
```

Static route updates

The `RouteStaticDirectory` class has been deprecated in favor of `StaticRoute.directory()`:

Before:

```
pod.webServer.addRoute(  
  RouteStaticDirectory(  
    serverDirectory: 'web/static',  
    basePath: '/',  
  ),  
  '/static/**',  
);
```

After:

```
pod.webServer.addRoute(  
  StaticRoute.directory(Directory('web/static')),  
  '/static/',  
);
```

The new `StaticRoute` provides better cache control options. You can use the built-in static helper methods for common caching scenarios:

```
pod.webServer.addRoute(  
  StaticRoute.directory(  
    Directory('web/static'),  
    cacheControlFactory: StaticRoute.publicImmutable(maxAge: 3600),  
  ),  
  '/static/',  
);
```

Available cache control factory methods:

- `StaticRoute.public(maxAge: seconds)` - Public cache with optional max-age
- `StaticRoute.publicImmutable(maxAge: seconds)` - Public immutable cache with optional max-age
- `StaticRoute.privateNoCache()` - Private cache with no-cache directive
- `StaticRoute.noStore()` - No storage allowed

You can also provide a custom factory function:

```
pod.webServer.addRoute(  
  StaticRoute.directory(  
    Directory('web/static'),  
    cacheControlFactory: (ctx, fileInfo) => CacheControlHeader(  
      publicCache: true,
```

```

        maxAge: 3600,
        immutable: true,
    },
),
'/static/',
);

```

Session.request property

With the Relic framework integration, the `Session` object now provides access to the underlying HTTP request through the `request` property. This allows endpoint methods to access request metadata such as client IP address and headers directly from the session.

Use optional chaining as it's `null` on some session types (e.g., sessions created for background tasks).

```

class MyEndpoint extends Endpoint {
  Future<String> getClientInfo(Session session) async {
    final ip = session.request?.remoteInfo ?? 'unknown';
    final userAgent = session.request?.headers.userAgent ?? 'unknown';
    return 'IP: $ip, UA: $userAgent';
  }
}

```

Authentication

Serverpod 3.0 includes several changes to the authentication system that improve type safety and performance.

NOTE

Besides minor interface changes described in this section, the legacy authentication system is still supported and can be used alongside the new version. This allows to safely upgrade your project to Serverpod 3.0 while using the legacy authentication and gradually migrate to the new one.

Custom authentication handlers

In the new authentication system, the default authentication header has changed from `Basic` to `Bearer` - which is now officially supported. This introduced a change for custom `AuthenticationHandler` implementations: the `token` parameter will now receive the token unwrapped from the `Bearer` prefix - just as it does for `Basic` tokens.

If your application relies on the default `authenticationHandler` function, no change is required. Only custom implementations that previously implemented `Bearer` authentication will need to be updated to expect the unwrapped token.

Another change when using custom authentication handlers is that tokens without scheme prefix are no longer supported by default to enforce HTTP header standards. The preferred fix is to wrap the token with the `Bearer` prefix, but this will invalidate existing tokens. If you are using a custom authentication handler in production and need to keep the old behavior, you can set the `validateHeaders` parameter to `false` in your `production.yaml` config file.

```
apiServer:
  ...

insightsServer:
  ...

webServer:
  ...

validateHeaders: false # ← Add this line

sessionLogs:
  consoleEnabled: false
```

AuthenticationInfo changes

The `AuthenticationInfo` class has been updated:

- `authId` is now non-nullable (previously optional)
- `userIdentifier` parameter type changed from `Object` to `String`

session.authenticated is now synchronous

Authentication is now resolved when the session is created, making `session.authenticated` synchronous. This improves performance by eliminating repeated async lookups.

Before (async):

```
final auth = await session.authenticated;
```

After (sync):

```
final auth = session.authenticated;
```

Client auth key provider

The `authKeyProvider` interface replaces the previous `authenticationKeyManager`. This interface has been simplified to make it more explicit what the client needs. It now only requires something that can provide an auth key wrapped as a header.

Before:

```
Client(host)..authenticationKeyManager = myManager;
```

After:

```
Client(host)..authKeyProvider = myProvider;
```

NOTE

To keep backwards compatibility with the old authentication system, the `FlutterAuthenticationKeyManager` from the legacy package now implements the `AuthKeyProvider` interface, so you can safely pass it to the client as the `authKeyProvider`.

Enum serialization

The default enum serialization strategy has changed from `byIndex` to `byName`. This change improves robustness when reordering or adding enum values, as serialized data remains valid even if the enum definition changes. With `byName`, the string representation is stored instead of the numeric index, making your data more resilient and easier to debug.

Keep old behavior:

```
enum: UserRole
serialized: byIndex # Add this
values:
  - admin
  - user
```

Use new default:

```
enum: UserRole
# serialized: byName - is now default
values:
  - admin
  - user
```

Model changes

SerializableEntity removed

The `SerializableEntity` class, deprecated since Serverpod 2.0, has been removed. Replace `extends SerializableEntity` with `implements SerializableModel` in your custom model classes.

Before:

```
class CustomClass extends SerializableEntity {  
  Map<String, dynamic> toJson() => {'name': name};  
}
```

After:

```
class CustomClass implements SerializableModel {  
  Map<String, dynamic> toJson() => {'name': name};  
  factory CustomClass.fromJson(Map<String, dynamic> json) => CustomClass(json['name']);  
}
```

Removed YAML keywords

The following deprecated YAML keywords have been removed. Run `serverpod generate` to see errors with migration guidance.

Old keyword	Replacement
<code>parent=table</code>	<code>relation(parent=table)</code>
<code>database</code>	<code>scope=serverOnly</code>
<code>api</code>	<code>!persist</code>

Before:

```
class: Company  
table: company  
fields:  
  name: String  
  ceoId: int, parent=employee  
  internalNotes: String, database  
  tempData: String, api
```

After:

```
class: Company  
table: company
```

```
fields:
  name: String
  ceoId: int, relation(parent=employee)
  internalNotes: String, scope=serverOnly
  tempData: String, !persist
```

Deprecated APIs

The following APIs have been deprecated but will continue to work for the foreseeable future to maintain compatibility with older clients. They will be removed in a future major version:

- Legacy streaming endpoints → Use [streaming methods](#) for new code

Other changes

Additional improvements in Serverpod 3.0:

- **Session log user ID as String:** The session log table now stores authenticated user IDs as `String` instead of `int`. This enables support for non-numeric identifiers such as UUIDs. A database migration is required (see [Create and apply migrations](#)).
- **Graceful SIGTERM shutdown:** The server now handles `SIGTERM` signals gracefully, allowing in-flight requests to complete before shutting down. This improves behavior in containerized deployments (Docker, Kubernetes) where orchestrators send `SIGTERM` before terminating processes.
- **Auto-stop on integrity check failure:** In development mode, the server now automatically stops if the database integrity check fails (e.g., schema mismatch). This prevents running with an inconsistent database state during development.

Resources

- [Relic Framework](#)
- [GitHub Issues](#)
- [Discord Community](#)